

4 The client SSL/TLS scanner

In the following chapter, we will discuss the development of a new tool, that is intended for the automated SSL/TLS testing, but from the different perspective. We already saw, in the previous chapters, that there are currently existing tools for the SSL/TLS testing, but all of them are during the encrypted communication used to test a remote server. A client, as well as the server, acts as an important party in the SSL/TLS communication. The question is therefore offered, why not test both parties? Therefore, we will now focus on the client's inclusion to the automated SSL/TLS testing. The tool for such a testing will be referred as the *Client scanner* in the rest of this thesis.

4.1 The functional requirements

First at all, we need to gather all the requirements:

- The support for the suitable tests.
- The simple configuration, that validates the user input and warns about the typos.
- The easily interpretable output.
- The application should be easily extended.
- Runnable on the Linux operating systems.
- Suitable for the automated testing.

4.1.1 The suitable tests

To test the client, we will examine communication, which originates from it, together with the general behavior of the client. Specifically, we want to perform the following tests:

- The test for the support of the specified cipher suites.
- The test for the behavior with the specified SSL/TLS protocols.
- The test for the behavior with the specified server certificates.

4.2 The software design

The implementation of the introduced tests requires full access to the client's communication with the server, together with the ability to intercept and potentially alter communication's content. For that reason, the Client scanner acts as the proxy server placed between the client and the remote server. The client scanner leaves communication untouched in the direction toward to the server, but may alter server's responses targeted for the client.

4.2.1 The configuration

The configuration is necessary to achieve a proper behavior as the proxy server. The same applies for the ability to make the decision, which client's behavior is considered as secure.

The configuration file configuration.xml

The file CONFIGURATION.XML, which is the main way to configure the Client scanner, consists of the four main categories:

- *directives* - Defines the necessary configuration directives for the proxy server. Specification of the local port, remote port and remote host is required for proper proxying functionality.
- *protocols* - Defines all the supported SSL/TLS protocols and for the each of them expected behavior.
- *cipherSuites* - Defines the selected cipher suites together with the expected behavior.
- *certificates* - Defines all the certificates, that will be available for the automated testing, including the expected behavior.

Code 4.1: The example content of the configuration.xml file

```
<configuration>
  <directives>
    <directive name="localPort" value="2222" />
    <directive name="remoteHost" value="https://domain.tld/" />
    <directive name="remotePort" value="443" />
  </directives>
</configuration>
```

```
</directives>

<protocols>
  <protocol name="SSLv2" mode="mustNotBe" />
  <protocol name="SSLv3" mode="mustNotBe" />
  <protocol name="TLS_1.0" mode="canBe" />
  <protocol name="TLS_1.1" mode="mustBe" />
  <protocol name="TLS_1.2" mode="mustBe" />
</protocols>

<cipherSuites>
  <cipherSuite
    name="TLS_ECDHE_RSA_WITH_RC4_128_SHA"
    mode="mustBe" />
</cipherSuites>

<certificates>
  <certificate
    name="endcert" mode="mustBe"
    path="file.jks" password="password" />
</certificates>
</configuration>
```

The file format of the certificates

From several reasons, the Java KeyStore was selected as the file format for the definition of the certificates. There currently exists tools, such as KeyStore Explorer [33], intended for the comfortable administration of this file format, which can be considered as a step to the easier usage of the Client scanner. KeyStore may further serve as the storage for the multiple certificates to create a whole certificate chain. The KeyStore is usually easily identified thanks to its *.jks file extension.

The definition of the cipher suites

To define the expected behavior regarding the cipher suites, a user of the Client scanner has to correctly state its name in the configuration file. Because there are currently the several naming conventions being used, the list of the approved names was created. Such the list is the internal part of the Client scanner. It may be altered, but this kind of the operation is intended for the Client scanner's administrator or

4. THE CLIENT SSL/TLS SCANNER

developer, not a regular user. Therefore rebuilding of the application from source codes, with the updated list of the approved cipher suites, is required. The mentioned list is located directly in the source codes (/SRC/MAIN/RESOURCES/CIPHERS.XML) in the form of XML file. Editing of the existing cipher suites requires inserting of the selected name and the proper cipher suite's hexadecimal code (may be found in the appropriate RFC).

Code 4.2: The example content of the ciphers.xml file

```
<cipherSuites>
  <cipherSuite
    hexValue="cca8"
    name="TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256"
  />
</cipherSuites>
```

4.2.2 The proxy server with the man-in-the-middle approach

The client scanner behaves as the proxy server. That means, that all the communication is going through the Client scanner, which allows to passively examine selected aspects of the SSL/TLS protocols or actively alter the communication. The man-in-the-middle approach is required to preserve access to the content of the communication. Without it, the content would be unreadable by the nature of the SSL/TLS protocols, that are obviously used to secure the communication's content before a third party.

The tested application has to always connect to the Client scanner. This connection will be secured by the certificate specified in the configuration. The second necessary connection (between the Client scanner and the remote server) is secured via the certificate provided by the remote server. Thanks to this approach, the traffic between the tested application and the remote server is intercepted. The Client scanner supplies the certificate to the tested application and impersonates the remote server. Then, the traffic received back from the secured remote server is provided to the tested application, but optionally with the certain modifications. This approach will enforce the tested application react to the occurred state (for example usage of the outdated SSLv2 protocol). The Client scanner then checks the tested application response and in the case of the unsafe behavior (for

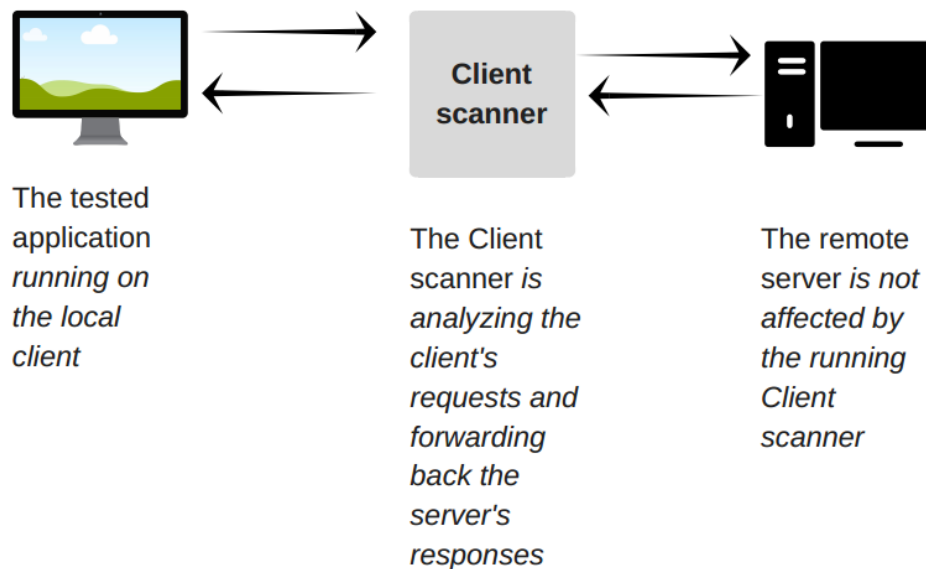


Figure 4.1: The simplified schema of the man-in-the-middle approach

example the tested application is communicating via the outdated SSLv2 protocol), such the discovered possible vulnerability is later on reported to the user.

The man-in-the-middle approach does not require the significant changes on the tested application side. The two main conditions must be fulfilled:

- The tested application, during its automated testing, must be deliberately communicating with the remote server. In the case of the connection closure (either caused by the application or the remote server), a new connection must be established by the tested application.
- At least one of the certificates supplied by the Client scanner, must be considered as trusted by the tested application. In practice, such the certificate may be self-signed or issued by the self-signed certificate authority. In both cases, the tested application must treat the certificated in the required way.

4. THE CLIENT SSL/TLS SCANNER

The first specified condition ensures, that thanks to the ongoing communication, all the required test may be performed. The second mentioned condition is rather required due to the ability to successfully intercept the communication with the man-in-the-middle approach.

4.2.3 The test for the specified cipher suites

The following test is implemented as the examination of the Client Hello message, that is being sent during the early stage of the SSL/TLS handshake. The mentioned message contains the collections of the cipher suites, that is the client willing to use to secure communication. The implemented test checks, if the message contains the expected cipher suites and the unspecified ones, usually considered as a weak, are not presented.

4.2.4 The test for the supported protocols

During this type of the testing, the client scanner is gradually changing its behavior to simulate the SSLv2, SSLv3, TLS 1.0, TLS 1.1 or TLS 1.2 protocol. A user of the Client scanner may specify, if the communication over each protocol must be successful or not.

4.2.5 The test for the server certificate

The last type of the test allows a user of the Client scanner to serve any server certificate during the SSL/TLS handshake. The mentioned server certificate may be valid or intentionally has a security issue to test the client behavior.

4.2.6 Exporting results to the HTML file

After performing all the necessary tests, easily interpretable output in the HTML form is created.

SSL/TLS analysis 2017-09-04 07:49:05 to 2017-09-04 07:50:17

Client Hello

Category	Mode	Status
Cipher suites	must not be	Cipher suite [TLS_RSA_WITH_AES_256_CBC_SHA (0x0035)] MUST NOT BE supported, but is present in Client Hello!

Certificate

Category	Mode	Status
Protocols	must be	Protocol [TLSv1.1] MUST BE supported, but there wasn't captured any communication to determine!

Figure 4.2: The example output of the Client scanner

4.3 The support for continuous integration

The Client scanner, just like Server scanner described in the previous chapter, comes with the support for continuous integration. Its usage can be automated and therefore it fits into the workflow of the continuous integration. Firstly, a single configuration file must be prepared, which is a one-time-only task. Then, the Client scanner can be repeatedly executed. The Server scanner always returns a boolean value after performing all tests. The mentioned boolean value indicates if tests successfully passed or not and therefore suggests if the developer should continue in the deployment or not. In the case of discovery of the possible security issues, a detailed output is created, which guides developer during the process of resolving the reported issues.

4.3.1 The environment with the multiple clients or servers

The single instance of Client scanner is meant to be used with a single executed tested application (in the client role). In the environment with multiple applications, that can be tested simultaneously, the Client scanner requires one configuration files per tested application. The configuration files must contain the mutually different TCP ports for proxying the traffic. Then, the single executable file of Client scanner can be run multiple times (with different configuration files) and therefore create multiple instances, which are testing individual applications. For that reason, Client scanner comes with the optional

console parameter that defines, which configuration file will be used (details are available in B.4). The Client scanner is not affected by a number of servers, which is tested application internally using to communicate. All traffic (from all servers) is going through the proxy. The case with multiple servers does not require additional configuration.

4.4 The testing

To test the Client scanner, some of the ordinary web browsers were used in the client's role. Then, the selected remote websites were repeatedly loaded using HTTPS protocol, but thanks to the Client scanner, the SSL/TLS communication was intercepted and altered in that ways, that the correct behavior of the implemented tests could be verified.

Some of the modern web browsers already have implemented more or less sophisticated tests for the unsafe server's behavior. A number of the intentionally created security risks was detected by the browser itself. The Client scanner passed, if the same security risk was correctly identified. In the remaining cases, the manual approach was used to verify the correct output of the Client scanner.

Some of the tests were repeated with the utility named cURL, which is a command-line application, that can be used for transferring the data via HTTPS. The mentioned utility offers various options to alter its behavior as the client (for example a user can specify the SSL/TLS protocol allowed for communication). In this case, the Client scanner passed, if successfully detected (intentionally created) unsafe settings on the cURL's side.